Practice Problems:
Foundations of Machine Learning

Instructor: Saketh
TAs: Nimai Parsa, Shravan Badgujar, Shreyas Shreekant Premkhede

November 23, 2025

Contents

Week 1

Bayes Optimal

1. Find the Bayes Optimal function corresponding to:

   (a) $\mathcal{X} = \mathbb{R}^n, \mathcal{Y} = \mathbb{R}^d$ and the underlying likelihood $p^*(x, y)$ is a Gaussian, with mean $\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}$ and inverse-covariance (precision matrix) $\begin{bmatrix} \Lambda_{11} & \Lambda_{12} \\ \Lambda_{12}^\top & \Lambda_{22} \end{bmatrix}$, where $\mu_1 \in \mathbb{R}^n, \mu_2 \in \mathbb{R}^d, \Lambda_{11} \in \mathbb{R}^{n \times n}, \Lambda_{12} \in \mathbb{R}^{n \times d}, \Lambda_{22} \in \mathbb{R}^{d \times d}$. Assume the loss is squared loss. Your answer must be a simplified expression involving these mean (sub)vectors and precision (sub)matrices only. Now using Schur-complement lemma re-write your answer replacing the precision sub-matrices with submatrices of covariance, $\begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{12}^\top & \Sigma_{22} \end{bmatrix}$. Does your final answer involve Σ_{11}? If not, is it surprising that it's not needed for the Bayes optimal computation?

   (b) $\mathcal{X} = \mathbb{R}, \mathcal{Y} = \{0, 1\}$ and $p^*(1/x) = \frac{1}{1+e^{x^2-5x+6}}$ and 0-1 loss.

   (c) $\mathcal{X} = \mathbb{R}, \mathcal{Y} = \{0, 1\}$ and $p^*(1/x) = \frac{1}{1+e^{x^2-5x+6}}$ and loss defined by $l(0, 0) = l(1, 1) = 0$ and $l(0, 1) = 0.5, l(1, 0) = 2$.

   (d) $\mathcal{X} = \mathbb{R}^n_+, \mathcal{Y} = \mathbb{R}_{++}$ and $\Lambda(x) = maxeig(\sum_{i=1}^n A_i x_i + A_0),\ x \geq 0$, where $A_1, \ldots, A_n$ are some psd (positive semi-definite) matrices, A_0 is a pd (positive definite) matrix and $maxeig$ is the maximum Eigen-value function. Consider square loss and $p^*(y/x) \propto e^{-\Lambda(x)y}, y > 0$.

(e) $\mathcal{X} = \mathbb{R}^n_+, \mathcal{Y} = \mathbb{R}_{++}$ and $\Lambda(x) = maxeig(\sum_{i=1}^n A_i x_i + A_0),\ x \geq 0$, where $A_1, \ldots, A_n$ are some psd (positive semi-definite) matrices, A_0 is a pd (positive definite) matrix and $maxeig$ is the maximum Eigen-value function. Consider $p^*(y/x) \propto e^{-\Lambda(x)y}, y > 0$ and absolute deviation loss defined by $l(y, z) \equiv |y - z|$.

Week 2

Linear Regression

1. Exercises 9.1, 9.2 in [Shalev-Shwartz and Ben-David(2014)].

2. Consider a regression problem where the label space is R^d. Consider the natural extension of the linear model studied in lecture: functions $f : \mathbb{R}^n \mapsto \mathbb{R}^d$ of the form $f(x) \equiv W^\top \phi(x)$, where W is a $n \times d$ matrix. Show that performing linear regression with squared-loss[1] using this model is essentially SAME as solving d classical linear regression problems over real labels. Hint: Exchange min-sum like in lectures.

3. Consider the above problem, but now with a "Mahalonabis loss": $l_\Sigma(y, z) \equiv (y - z)^\top \Sigma (y - z), \Sigma \succ 0$. Observe that if Σ is the identity matrix, then this is same as the familiar squared loss. Show that this linear regression problem is same as solving d classical linear regression problems over modified labels: $\tilde{y} \equiv \Sigma^{-\frac{1}{2}} y$.

2.1 Planetary Weather Prediction Challenge – Mathematical Simulation

2.1.1 Scenario

You are part of a research mission on the distant planet **Exo-7**. Each day, you collect a vector of atmospheric readings from d different sensors. Sensor measurements could include sunlight levels, wind speed, cloud cover,

[1] $l_2(y, z) \equiv \|y - z\|^2$.

and gas composition. Your task is to **predict tomorrow's average temperature** from today's sensor data.

2.1.2 Data Generation

1. **Feature Vectors (Sensor Readings)** Assume the d-dimensional sensor readings are drawn as:

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}),$$

where $\boldsymbol{\mu} \in \mathbb{R}^d$ is any chosen mean vector and $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ is a positive-definite covariance matrix.

2. **True Climate Law** Let the "true" model parameters be a vector:

$$\boldsymbol{\theta}_{\text{climate}} \in \mathbb{R}^d.$$

For a given day's sensor readings $\mathbf{x}$, the average temperature tomorrow is generated as:

$$y \sim \mathcal{N}\left(\boldsymbol{\theta}_{\text{climate}}^\top \mathbf{x},\ \sigma^2\right),$$

where σ^2 is the variance of random noise (e.g., $\sigma^2 = 1$).

3. **Dataset Size and Splitting** Generate $2m$ independent $(\mathbf{x}, y)$ pairs. Randomly assign m of them to the *training set* (D) and the remaining m to the *test set* (T).

2.1.3 Learning Task

1. **Bayes Optimal Parameter** Show that in the above setting, the Bayes optimal linear predictor for y given $\mathbf{x}$ is:

$$\hat{y} = \boldsymbol{\theta}_{\text{climate}}^\top \mathbf{x}.$$

2. **Parameter Estimation** Using the training set D, estimate $\boldsymbol{\theta}_{\text{climate}}$ by solving the *ordinary least squares* problem:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \sum_{(\mathbf{x}_i, y_i) \in D} \left(y_i - \boldsymbol{\theta}^\top \mathbf{x}_i\right)^2.$$

You may:

- Solve the *normal equation*:

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

  using `numpy.linalg.lstsq`, OR

- Use a `LinearRegression` implementation from `scikit-learn`.

3. **Performance Evaluation** For the test set T, compute the *explained variance score*:

$$\mathrm{EV}(T) = 1 - \frac{\sum_{(\mathbf{x}_i, y_i) \in T} (y_i - \hat{y}_i)^2}{\sum_{(\mathbf{x}_i, y_i) \in T} (y_i - \bar{y}_T)^2},$$

   where $\hat{y}_i$ is the model's prediction on $\mathbf{x}_i$, and $\bar{y}_T$ is the mean of y values in T.

2.1.4 Experiment

Keep d fixed, say, $d = 10$, and repeat the above experiment while varying the number of samples m (e.g., $m = 15, 25, 35, 45, \ldots$).

- For each m, record $\mathrm{EV}(T)$.
- Plot **Explained Variance vs.** m.
- If the solver fails for some m (e.g., due to singular matrix), report those cases.

Keep $m = 200$ fixed and repeat the above experiment while varying the number of sensors d (e.g., $d = 2, 5, 10, 20, \ldots$).

- For each d, record $\mathrm{EV}(T)$.
- Plot **Explained Variance vs.** d.
- If the solver fails for some d (e.g., due to singular matrix), report those cases.

2.2 Effect of Feature Correlation, Noise Level, and Sample Size on Linear Regression

2.2.1 Objective

Investigate how feature correlation, label noise variance, and training sample size jointly affect the performance of linear regression compared to the Bayes optimal predictor.

2.2.2 Data Generation

Fixed Parameters:

- Dimension: $d = 20$
- True parameter vector:

$$w^* = (1, 1, \ldots, 1) \in \mathbb{R}^d$$

Feature Distribution: Generate $X \in \mathbb{R}^{n \times d}$ from a multivariate Gaussian distribution with covariance matrix:

$$\Sigma_{ij} = \rho^{|i-j|}$$

where $\rho \in [0, 1)$ is the *feature correlation parameter*. For each dataset, choose a fixed ρ.

Labels with Noise: For each sample x_i, generate:

$$y_i = w^{*T} x_i + \epsilon_i$$

where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ and σ^2 is the *noise variance*, varied in experiments.

Bayes Optimal Predictor: The Bayes optimal linear predictor is w^*. For a given σ^2, the minimum achievable MSE is:

$$\text{MSE}_{\text{Bayes}} = \sigma^2$$

2.2.3 Experiments

For each combination of:

$$\rho \in \{0, 0.5, 0.9\}, \quad \sigma^2 \in \{0.01, 1, 5\}, \quad m \in \{50, 200, 1000\}$$

perform the following steps:

1. Generate $n = 2m$ samples as described above.
2. Randomly split into:
   - Training set D of size m
   - Test set T of size m
3. Fit linear regression using `numpy.linalg.lstsq` or `sklearn.linear_model.LinearRegression` on D.
4. Evaluate test MSE on T.
5. Record Bayes MSE ($= \sigma^2$) for comparison.

2.2.4 Outputs Required

- **Plot 1:** Test MSE vs m for each ρ (different noise levels in separate subplots).
- **Plot 2:** Test MSE vs ρ for each m (different σ^2 in separate subplots).
- **Table:** Gap between achieved MSE and Bayes MSE for each configuration.

2.2.5 Questions to Answer

1. How does high feature correlation ($\rho \approx 1$) affect estimation error for small m? Why?
2. Does increasing sample size always close the gap to the Bayes MSE? Explain.
3. For fixed m, does increasing σ^2 make correlation effects more or less important? Why?

4. Compare the empirical results to the theoretical Bayes limit — in which scenarios do you achieve near-optimal performance?

Week 3

Linear (Binary) Classification

Assume: $\mathcal{X} = \mathbb{R}^n, \mathcal{Y} = \{-1, 1\}$.

1. Consider a problem where the underlying likelihood is given by $p(1|x) = \frac{(x-3)^2}{(B+3)^2}$ and $p(x)$ is uniform [-B,B]. Write the optimization problems defining the Bayes optimal linear classifiers over the feature map $\phi(x) = \begin{bmatrix} x \\ 1 \end{bmatrix}$ with 0-1 loss and hinge loss (unregularized).
   (a) In special case $B = 3$, solve these to write simplified expressions for the Bayes optimal linear classifiers.
   (b) What is the model error for these cases ? Provide exact values.
   (c) For what values of $B > 0$ is the model error exactly zero ?
   (d) For what values of $B > 0$ is the optimal linear classifier a constant (always predicts the same label)?
2. Consider a problem where the underlying likelihood is defined by $p(x|i) \sim \mathcal{N}(\mu_i, \Sigma_i), i = 1, -1$. $p(1) = 0.4$. Firstly, using Bayes rule, derive a simplified expression for the posterior. Derive necessary and sufficient conditions om the covariances Σ_1, Σ_2 such that the model error with linear model over the ϕ defined in earlier problem is exactly zero. Analogously derive conditions on means and covariances such that model error with linear model over $\mathcal{X}$ is exactly zero.
3. Consider a binary classification problem with the training data:
$$\mathcal{D} = \left\{ \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, 1 \right), \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}, -1 \right), \left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, -1 \right), \left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}, 1 \right) \right\}$$

and the feature map ϕ is defined by

$$\phi\left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix}\right) \equiv ((z_1 + z_2)\%2) - \frac{1}{2} \; \forall \; z_1, z_2 \in \mathbb{R}.$$

Here, $a\%b$ is the remainder when a is divided by b. For this case, write down a simplified expression for the ERM classifier.

Week 4

Gradient Descent

1. Show that the risk minimization objective with linear models is smooth whenever the loss is itself smooth.

2. Show that the risk minimization objective with linear models is convex whenever the loss is itself convex.

Problem 1 — Playing with Step Size in Gradient Descent (Linear Regression)

We have implemented gradient descent for you inside a black-box function `gradient_descent(...)`. Your job is to study how the step size (learning rate) affects the learning process for linear regression.

Data Generation

First, you need to generate the synthetic dataset.

1. Generate $N = 2000$ samples from a d-dimensional multivariate Gaussian with mean vector 0 and variance 1, also generate 2000 labels for corresponding data.

2. Choose your own weight vector $\mathbf{w} \in \mathbb{R}^d$ and bias vector $\mathbf{b} \in \mathbb{R}$.

3. Generate the corresponding predicted labels y_i using the following linear model with additive Gaussian noise:

$$y = \mathbf{w}^T\mathbf{x} + b$$

Training

Proceed with the following training and evaluation steps.

1. Split your generated dataset of $N = 2000$ samples into a training set (80% of the data) and a testing set (20% of the data).

2. You will use the provided gradient descent solver to fit a linear regression model. The goal is to find a weight vector $\mathbf{w}$ that minimizes the mean squared error.

3. Run the solver separately for each of the following step sizes (learning rates), η:

$$\eta \in \{0.0001, 0.001, 0.01, 0.1\}$$

   Ensure the maximum number of iterations is fixed for all runs (e.g., 500 iterations). The function should record the training loss and testing loss at each iteration.

Tasks

1. For each of the four values of η, create a single plot that shows both the training loss and the test loss as a function of the iteration number. Your plot should have "Iteration Number" on the x-axis and "Loss" (e.g., Mean Squared Error) on the y-axis. Use a legend to distinguish between the training and testing curves.

2. Based on your plots, provide a detailed commentary on the effect of the step size. Your analysis should address the following points:

   - **Convergence Speed:** Which value of η converges to a low error the fastest?
   - **Divergence:** Does any value of η cause the loss to increase, indicating that the algorithm is diverging? How is this visible in the plot?

- **Underfitting/Slow Convergence:** Does any value of η result in very slow convergence or convergence to a suboptimal (high) loss value?

Week 5

Stochastic Gradient Descent

Problem 7 — Empirical Comparison of Scikit-Learn Optimizers

Overview

You are tasked with evaluating and comparing four different optimization algorithms available in `scikit-learn` for training a logistic regression model. The objective is to analyze their performance in terms of convergence speed over epochs, scalability with respect to the number of samples (n), and final model accuracy.

Dataset

You will use synthetic binary classification datasets generated via `sklearn.datasets.make_classific`
The data should be split into 80% for training and 20% for testing, using a fixed random state for reproducibility.

1. **For Scalability & Convergence Analysis:** Generate datasets with $p = 20$ features (10 informative) and a varying number of samples $n \in \{500, 5000, 20000\}$.

2. **For Visualization:** Generate a separate dataset with $n = 1000$ samples and $p = 2$ features (both informative) to visualize the optimization paths.

Algorithms and Implementation

You will use `scikit-learn`'s `LogisticRegression` and `SGDClassifier` models to compare the following three solvers: SGD, SAG, and SAGA.

Experiments

Part 1: Convergence and Scalability Analysis For each sample size $n \in \{500, 5000, 20000\}$:

1. Generate the dataset and split it.

2. For each of the three algorithms, train the model for a total of **100 epochs**.

3. To track performance at each epoch, you will train the model one epoch at a time. For `LogisticRegression` solvers, set `warm_start=True` and `max_iter=1` and call `fit` in a loop. For `SGDClassifier`, use the `partial_fit` method.

4. Record the training and test loss after each epoch.

5. Record the total training time for the full 100 epochs.

6. After 100 epochs, evaluate the final model's classification accuracy on the test set.

Part 2: Optimization Path Visualization Using the 2D dataset ($p = 2$):

1. Create a contour plot of the logistic loss surface.

2. For each of the three optimizers, trace the actual optimization path of the weights (θ) over 50 iterations using the same incremental fitting technique from Part 1.

3. Overlay these paths on the contour plot to visualize their behavior.

Outputs Required

- **Plots 1-3:** A separate plot for each value of $n \in \{500, 5000, 20000\}$. Each plot must show the **Training Loss and Test Loss vs. Epochs** for all three optimizers. The y-axis should be on a logarithmic scale.
- **Plot 4:** A single plot of **Training Time (s) vs. Number of Samples (n)** for all three optimizers.
- **Plot 5:** The contour plot of the loss surface for the 2D dataset, showing the actual optimization path for all three optimizers.
- **Table 1:** A summary table presenting the Training Time (s), Final Test Loss, and Final Test Set Accuracy (%) for each optimizer on the largest dataset ($n = 20000$).

Week 6

Likelihood estimation

1. Simplify MLE problems for multivariate Gaussian model, categorical/multinoulli model to obtain analytical expressions. For e.g., multivariate Gaussian is given in section 4.2.6 in [Murphy(2022)] and Bernoulli/multinoulli in section 4.2.3/4.2.4 of the same book. Repeat same for Poisson model, Exponential distribution, Gamma model.

2. Exercises 4.5a-c, 4.7 in [Murphy(2022)].

3. Show that multivariate Gaussian model, Bernoulli, categorical/multinoulli model belong to the exponential family. For e.g., see section 3.4.2 in [Murphy(2022)]. Repeat the same for Poisson model, Exponential distribution, Gamma model. Clearly identify the sufficient statistics, express natural parameters in terms of usual parameters and vice-versa. Identify the base likelihood functions.

4. Consider the sufficient statistics: $\phi(y) \equiv \begin{bmatrix} y^2 \\ y^4 \end{bmatrix}$. Assume base likelihood is unity. Compute a simplified expression for the partition function.

Week 7

Probabilistic Models for Supervised Learning

Assume: Training data is training dataset $\mathcal{D} = \{(x_1, y_1), \ldots, (x_m, y_m)\}$.

7.1 Generative models

1. Consider the generative linear regression presented in lecture and assume $\hat{\mu}_1, \hat{\mu}_2$ are not zero. Consider a (non-probabilistic) linear regression model with $\phi(x) = \begin{bmatrix} x \\ 1 \end{bmatrix}$ trained on exactly the same data. Will the final regressors with these two models be the same (after training) ? Recall from lectures that these are the same if $\hat{\mu}_1, \hat{\mu}_2 = 0$ and $\phi(x) = x$.

2. Consider the generative linear regression presented in lecture. Can the final regressor in this case decouple across output dimensions ? i.e., is the generative linear regressor obtained by considering training data for each output dimension separately is the same as the generative linear regressor, trained on entire training data, for the corresponding output dimension ? Recall that from problem 2, this is true for (non-probabilistic) linear regression.

3. Consider a generative model for regression problems where $p^*(y)$ is modelled using a Gaussian model i.e., $\mathcal{N}(\mu_2, \Sigma_2)$, where $\mu_2 \in \mathbb{R}^d, \Sigma_2 \succ 0$ are the parameters. And, $p^*(x/y)$ is modelled using:

$\mathcal{N}(W^\top y + b, \Sigma_1)$, where $W \in \mathbb{R}^{d\times n}, b \in \mathbb{R}^n, \Sigma_1 \succ 0$ are the parameters. Show that $p^*(y/x)$ will be a Gaussian. Express this Gaussian's mean and covariance in terms for $\mu_2, W, b, \Sigma_2, \Sigma_1$. This model (for the joint likelihood $p^*(x, y)$) is called as the linear Gaussian system. Simplify the expression for Bayes optimal with this model. Simplify the MLE problem in this case. Is this model also plagued with the degeneration/decoupling issue in the previous problem?

7.2 Discriminative modelling

1. Consider a Poission regression model: $p(y|x)$ is modelled as a Poisson likelihood with mean $e^{w^\top \phi(x)}$. Present simplified expressions for the MLE solution. Present simplified expressions for the final predictive function with 0-1 loss, and with absolute deviation loss (this is more appropriate for this output space). Hint: Use Ramanujan equation `https://www.jstor.org/stable/2160389?seq=`.

2. Consider a generative model where label is a natural number (i.e., represents a count) and input is a scalar. Let class-conditional be modelled by a (univariate) Gaussian: $p(x|y) \sim \mathcal{N}(\mu y, \sigma^2 y^2)$ and prior $p(y)$ be modelled by a Poisson likelihood, with parameter λ. More specifically, $Y - 1$ is Poisson, as label is a natural number. Present simplified expressions for trained parameters with a training dataset. Write a simplified expression for the posterior likelihood with the trained parameters. You may skip normalization constants. Is this posterior a Poisson again? Present simplified expression for the final predictive function. Consider loss as 0-1 loss.

Week 8

Model Selection

This problem focuses on the practical application of k-fold cross-validation (CV) for hyperparameter tuning and observing the trade-offs in computational cost, bias, and variance associated with the choice of k.

Problem Setup: Hyperparameter Tuning with CV

Data and Model

1. **Data:** Use a standard binary classification dataset eg,. breast cancer data from the sklearn.
2. **Model:** Use the Stochastic Gradient Descent (SGD) Classifier from sklearn and set the loss function as ''log_loss'' for logistic regression.
3. **Preprocessing:** Ensure data is standardized (i.e., scaled) before training.

Hyperparameter Grid

Use the GridSearchCV module from sklearn to get the grid search on the following hyperparameters:

- **Regularization Parameter (α):** Test at least three values (e.g., $\{10^{-4}, 10^{-3}, 10^{-2}\}$).
- **Regularization Type (Penalty):** Test major types ({'l1', 'l2', 'elasticnet'}).

- **Learning Rate/Step-Size Strategy (η):** Test key strategies (e.g., {'constant', 'optimal', 'invscaling'}).
- **Optimization/Stopping Criteria:** Test the effect of **Early Stopping** ({True, False}).

Cross-Validation Strategy

Implement a routine (e.g., using `GridSearchCV`) to perform the grid search and select the best model based on accuracy. Define three specific CV scenarios for comparative analysis:

1. **CV Scenario 1 (High Variance):** $k = 2$.
2. **CV Scenario 2 (Standard):** $k = 5$ (or $k = 10$).
3. **CV Scenario 3 (Low Variance/High Cost):** Leave-One-Out Cross-Validation (LOOCV), where $k = N$ (the total number of samples).

Questions to answer

1. **Optimal Hyperparameters (Scenario 2):** Using the **Standard CV Scenario ($k = 5$ or $k = 10$)**, run the grid search once. Report the optimal set of hyperparameters found (α, Penalty, η, Early Stopping) and the corresponding mean cross-validation score.
2. **Computation Time:** Measure and report the total time required to run the full grid search (across all combinations in $\mathcal{G}$) for each of the three CV scenarios ($k = 2$, $k = 5/10$, and $k = N$). Explain the relationship you observe between the number of folds (k) and the computation time in terms of the total number of training runs performed.
$$\text{Total Training Runs} = k \times |\mathcal{G}|$$
3. **Variability and Robustness (The Core Insight):**
   (a) **Procedure:** Repeat the entire model selection process (the full grid search) **5 times** for both $k = 2$ and $k = N$ **(LOOCV)**. Ensure that the random split/shuffle of the data into folds changes for each repeat (e.g., by changing `random_state` in the `KFold` object).

(b) **Analysis:** Record the best hyperparameters and the CV score found in each of the 5 repeats for both $k = 2$ and $k = N$.

(c) **Observation:** Compare the **variability** (range or standard deviation) of the final selected hyperparameters and the CV scores between the $k = 2$ repeats and the $k = N$ repeats. Why is the variability generally higher with $k = 2$ and lower with LOOCV ($k = N$)? (Hint: Discuss the **size** and **overlap** of the training sets in each fold.)

4. **Bias-Variance Trade-off in CV:** Based on the results from the comparison in Question 3, explain the general **bias-variance trade-off** associated with the choice of k in k-fold CV. Specifically address:

   - How does a **smaller** k **(e.g.,** $k = 2$**)** affect the **bias** of the performance estimate?
   - How does a **larger** k **(e.g.,** $k = N$**)** affect the **variance** of the final model selection result?

Week 9

Kernel Methods

Either using the definition and/or using the theorems listed in the lecture, answer the following:

1. Is $k(x, y) \equiv \cos(x - y)$ a valid kernel over $\mathbb{R}$?

2. Is $k(x, y) \equiv \cos(h(x) - h(y))$ a valid kernel over $\mathbb{R}$ for any $h : \mathbb{R} \mapsto \mathbb{R}$?

3. Is $k'(x, y) \equiv \cosh(k(x, y))$ a valid kernel whenever k is a valid kernel?

4. Show that $k(X, Y) \equiv \int_{\mathcal{X}} \int_{\mathcal{X}} f_X(x) f_Y(y) h(x, y)$dxdy, where X, Y are random variables over $\mathcal{X}$ with corresponding densities f_X, f_Y; is a kernel whenever $h : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$ is a kernel. This observation is the key behind so-called kernel embeddings of likelihoods, which play a central role in modern statistical hypothesis testing methods.

SVM vs. Logistic Regression: A Kernel-Based Comparison

Overview

You are given the task to compare the performance of Support Vector Machines (SVM) and Logistic Regression on two synthetic datasets. The objective is to understand the strengths and weaknesses of each algorithm.

Datasets

You will work with two different binary classification datasets. For both, you should use an 80% training and 20% testing split.

Dataset 1: Non-Linearly Separable Data Use `sklearn.datasets.make_moons` to generate a dataset that is not separable by a straight line.

$$X_{\text{moons}} \in \mathbb{R}^{n \times 2}, \quad y_{\text{moons}} \in \{0, 1\}$$

Generate the dataset with $n = 500$ samples, add noise with a standard deviation of 0.25, and use a `random_state` of 42 for reproducibility.

Dataset 2: Linearly Separable Data Use `sklearn.datasets.make_classification` to generate a dataset that is largely linearly separable.

$$X_{\text{linear}} \in \mathbb{R}^{n \times 2}, \quad y_{\text{linear}} \in \{0, 1\}$$

Generate the dataset with $n = 500$ samples, 2 informative features, 0 redundant features, 1 cluster per class, `flip_y` $= 0$ and use a `random_state` of 42 for reproducibility.

Algorithms to Compare

You will train and evaluate the following models from the `scikit-learn` library:

1. **Logistic Regression**
2. **Support Vector Machine with a linear kernel**
3. **Support Vector Machine with a non-linear kernel**

Experiments

Step 1: Model Training and Evaluation For each of the two datasets, perform the following steps:

1. Train all three models on the training portion of the dataset.
2. For your non-linear SVM, use the default hyperparameters.
3. Evaluate the classification accuracy of each trained model on the corresponding test set.

Step 2: Visualization of Decision Boundaries To better understand how each model separates the data, you need to create visualizations.

1. For each dataset, create a single scatter plot showing the test data points, colored by their true class.

2. On this same plot, overlay the decision boundaries for all three trained models. Use different colors or line styles to distinguish the boundaries. A helper function to plot decision boundaries will be useful here.

Outputs Required

- **Plot 1:** A plot for the `make_moons` dataset showing the test data points and the decision boundaries for all three models.

- **Plot 2:** A plot for the linearly separable `make_classification` dataset showing the test data points and the decision boundaries for all three models.

- **Table 1:** A summary table presenting the Test Set Accuracy (%) for each of the three models on both datasets.

Questions to Answer

1. Based on your results for Dataset 1 (moons), which model performed best? By looking at its decision boundary plot, explain why it was more effective than the other two. Which non-linear kernel gave the best results compared to other kernels for this dataset?

2. Based on your results for Dataset 2 (linear), how did the performances of the three models compare? Why do you think this was the case?

Kernel Regression: Matching Model Bias to Function Smoothness

Overview

The goal of this problem is to understand how a model's underlying assumptions (its inductive bias) affect its ability to learn from different types of data. You will use Kernel Ridge Regression with Matern kernels of varying smoothness to model two synthetic datasets: one generated from an infinitely smooth function and another from a discontinuous function.

Functions and Datasets

You will work with two one-dimensional datasets. For both, you should generate $n = 15$ training samples.

Dataset 1: Smooth Function Data Generate data from a sine function with added Gaussian noise. The x values should be sampled uniformly from the interval $[-6, 6]$.

$$y_{\text{smooth}} = \sin(x) + \epsilon, \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, 0.3^2)$$

Use a `random_state` of 42 for reproducibility.

Dataset 2: Discontinuous Function Data Generate data from a sign function (a step function) with added Gaussian noise. The x values should also be sampled uniformly from $[-6, 6]$.

$$y_{\text{discontinuous}} = \text{sign}(x) + \epsilon, \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, 0.3^2)$$

Use the same `random_state` for reproducibility.

Model to Use

You will train and evaluate a single type of model, but vary its internal assumptions by changing the kernel.

(a) **Kernel Ridge Regression with Matern Kernels**: You will use a fixed implementation of Kernel Ridge Regression and test it with a series of Matern kernels, each defined by a different smoothness parameter, ν.

Experiments

Step 1: Model Training For each of the two datasets, perform the following steps:

(a) Train a separate Kernel Ridge Regression model for each value of the smoothness parameter $\nu \in \{0.5, 1.5, 2.5, 5.0, 10.0, 100.0\}$.

(b) Use a fixed **length-scale** of $l = 2.0$ for the Matern kernel across all experiments.

(c) Use a fixed **regularization parameter** of $\lambda = 10^{-3}$. The system to solve is $(\mathbf{G} + \lambda n \mathbf{I})\boldsymbol{\alpha} = \mathbf{y}$.

Step 2: Visualization of the Regression Fits To understand how the kernel's smoothness assumption affects the model, you will create two sets of visualizations.

(a) For each dataset, create a single figure containing a 2x3 grid of subplots. Each subplot will correspond to one value of ν.

(b) In each subplot, you must plot three things:

- The true underlying function (e.g., $\sin(x)$) as a dashed line.
- A scatter plot of the noisy training data points.
- The regression curve predicted by the trained model.

Outputs Required

- **Figure 1:** A 2x3 grid of plots for the **smooth function dataset** ($\sin(x)$), showing the results for all six values of ν.
- **Figure 2:** A 2x3 grid of plots for the **discontinuous function dataset** ($\text{sign}(x)$), showing the results for all six values of ν.

Questions to Answer

(a) For the smooth sine function, which values of ν appear to give the best fit to the true underlying function? Explain why a high value of ν is effective here.

(b) For the discontinuous sign function, describe how the model's behavior changes as ν increases. Pay close attention to the model's prediction around the discontinuity at $x = 0$.

(c) Why do all the models fail to capture the sharp jump in the sign function? Relate this failure to the concept of a model's **inductive bias**.

Week 10

Deep Learning

1. Exercise 13.1 in [Murphy(2022)].
2. Exercises 2-5 in [Shalev-Shwartz and Ben-David(2014)].
3. Exercises 1-4 in 3.10 in Charu's book.
4. Exercises 5.1,5.6-5.9,5.18 in Bishop's book.

Investigating the Impact of Network Depth on Overfitting

Objective The goal of this problem is to empirically study how increasing the depth (number of hidden layers) of a Multi-Layer Perceptron (MLP) affects its ability to fit training data and generalize to unseen test data. This experiment will highlight the concepts of underfitting, the trade-off between bias and variance, and how excessive model complexity can lead to overfitting.

Data Generation

1. You will use a synthetic dataset that is not linearly separable, making it a suitable task for a neural network. Use the `make_moons` function from `sklearn.datasets`.
2. Generate a dataset with the following parameters:
    - `n_samples = 1000`

- `noise = 0.3`
- `random_state = 42`

3. **Preprocessing**:
   - Split the generated data into a training set (70% of the data) and a test set (30% of the data). Use a fixed `random_state` for reproducibility.
   - Standardize the features using `sklearn.preprocessing.StandardScaler`. Fit the scaler on the training data and use it to transform both the training and test data.

Experiments You will train several `MLPClassifier` models from `scikit-learn` with varying depths. The width of the hidden layers will be kept constant to isolate the effect of depth.

1. **Architectures to Test**: You will train and evaluate a separate model for each of the following network depths:
   - $d \in \{1, 2, 3, 5, 10, 20\}$
   - For each depth d, the network will have d hidden layers, each containing 32 neurons. For example, for $d = 3$, the `hidden_layer_sizes` parameter would be `(32, 32, 32)`.
2. **Fixed Hyperparameters**: For all experiments, use the following fixed settings for the `MLPClassifier` to ensure a fair comparison:
   - `activation = 'relu'`
   - `solver = 'adam'`
   - `max_iter = 500`
   - `learning_rate_init = 0.001`
   - `random_state = 42`
3. **Evaluation**: For each trained model, you must record two metrics:
   - The final classification accuracy on the **training set**.
   - The final classification accuracy on the **test set**.

Outputs Required

1. **Plot**: Create a single plot that shows both the training accuracy and the test accuracy as a function of the network depth.
    - The x-axis should represent the "Network Depth (Number of Hidden Layers)".
    - The y-axis should represent "Classification Accuracy".
    - The plot must contain two distinct lines or series of points: one for training accuracy and one for test accuracy. Use a legend to distinguish between them.

2. **Table**: Create a summary table that presents the final results for each experiment. The table should have the following columns: "Network Depth", "Final Training Accuracy", and "Final Test Accuracy".

Questions to Answer

1. Based on your plot, describe the trend observed for the **training accuracy** as the network depth increases. Why does the model's performance on the data it was trained on behave this way?

2. Now, describe the trend for the **test accuracy**. At what depth does the model achieve the best generalization performance? What happens to the test accuracy as the models become very deep (e.g., $d = 10, 20$)?

3. Explain the divergence between the training and test accuracy curves for deeper networks. What is this phenomenon called, and why does it occur?

4. If you were to deploy one of these models for practical use, which architecture would you choose and why? Relate your choice to the bias-variance trade-off.

Week 11

Overparametrized Models

Double Descent in Random Features Model

In this problem, we explore the **double descent phenomenon** that appears in overparameterized models trained by gradient descent (GD). We will study this behavior through a controlled simulation using a **random features model**.

Data Generation

We consider an input dimension $d = 5$ and generate $n = 200$ data points $\{(x_i, y_i)\}_{i=1}^n$ as follows:

- Each input vector $x_i \in \mathbb{R}^d$ is sampled uniformly from the **unit sphere**:

$$x_i \sim \text{Uniform}(S^{d-1}) \quad \text{where} \quad S^{d-1} = \{x \in \mathbb{R}^d : \|x\|_2 = 1\}.$$

- A random direction $v_* \in \mathbb{R}^d$ is drawn uniformly from the unit sphere. This vector defines the "true" underlying target relationship.

- The output labels are generated according to the noisy nonlinear function

$$y_i = \frac{1}{4} + (v_*^\top x_i)^2 + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

  where $\sigma = 2$ controls the noise level.

Model: Random Features Regression

We build a random feature mapping $\phi_m : \mathbb{R}^d \to \mathbb{R}^m$ using m randomly initialized neurons:

$$\phi_m(x) = \Big((v_1^\top x)_+, (v_2^\top x)_+, \ldots, (v_m^\top x)_+\Big),$$

where each $v_j \in \mathbb{R}^d$ is sampled uniformly on the unit sphere, and $(z)_+ = \max(0, z)$ denotes the ReLU activation.

We then fit a **linear readout layer**

$$\hat{y}(x) = w^\top \phi_m(x) + b,$$

where $w \in \mathbb{R}^m$ and $b \in \mathbb{R}$ are trained parameters. The model is optimized using gradient descent to minimize the mean squared error (MSE) loss:

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(\hat{y}(x_i) - y_i\Big)^2.$$

Task

1. Implement the above model for varying numbers of random features $m \in \{10, 40, 70, 100, 130, \ldots, 1000\}$.
2. For each m, train the model using gradient descent until convergence.
3. Compute both the **training error** and **test error** (on a large independent test set, e.g. 1000 samples).
4. Plot the training and test mean squared error as a function of m.

Expected Observation

You should observe the following behavior:

- For small m (underparameterized regime), both training and test error are high.
- Around $m \approx n$, the model has enough capacity to interpolate the training data, resulting in near-zero training error but a sharp **peak in test error**.
- As m continues to increase ($m \gg n$), the test error decreases again, leading to the characteristic **double descent curve**.

Questions to Discuss

1. Why does the test error increase near the interpolation threshold ($m \approx n$)?
2. How does the noise variance σ^2 affect the height of the peak?

Week 12

Online Learning

- Exercise 4 in 21.7 in [Shalev-Shwartz and Ben-David(2014)].

Week 13

Boosting

1. Rederive the update equations for boosting with logistic regression loss for binary classification.
2. Repeat the same with multi-class classification.

Bibliography

[Murphy(2022)] Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL `probml.ai`.

[Shalev-Shwartz and Ben-David(2014)] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, New York, NY, USA, 2014. ISBN 1107057132, 9781107057135.